

ADVANCED PERSONAL FIREWALL WITH SYSTEM ENFORCEMENT

1. Introduction

This project details the development of a robust, lightweight Personal Firewall created using Python. The primary objective was to build a tool capable of performing real-time network traffic analysis, enforcing custom security rules, and proactively defending the host system by integrating with native operating system controls. The project successfully created a firewall that operates as both a sophisticated traffic monitoring tool and a kernel-level enforcement mechanism.

2. Tools Used

Core Languages/Libraries: Python 3, scapy (for packet manipulation), subprocess (for system command execution), datetime (for timestamping logs).

Operating System/Virtualization: Kali Linux (Guest OS), Oracle VM VirtualBox (Hypervisor).

System Controls: iptables (Linux kernel firewall control) and sudo (for necessary root privileges).

Editor: GNU nano.

3. Steps Involved in Building the Project

3.1 Packet Sniffer Development

The foundation of the firewall was built using the `scapy.sniff()` function to capture traffic on the `eth0` interface and extract packet headers. This provided real-time visibility into the network stream.

Code Snippet: Core Sniffing and Packet Processing Logic

```
from scapy.all import sniff, IP, TCP, UDP
```

```
from datetime import datetime
```

```
import sys
```

```
import subprocess # For running shell commands (iptables)
```

```
# -----
```

```
# 1. GLOBAL FIREWALL RULES
```

```

# -----

# Ports: Block Telnet (23) and HTTP (80)
BLOCKED_PORTS = [23, 80]

# Protocols: Block ICMP (1), which is used for the ping command
BLOCKED_PROTOCOLS = [1]

# IPs: Placeholder IP - CHANGE THIS to the IP of the machine you want to test blocking from.
BLOCKED_SOURCE_IPS = ["1.2.3.4"]


# Logging Configuration
LOG_FILE = "firewall_audit.log"
INTERFACE_NAME = "eth0" # Verified interface


# -----

# 2. HELPER FUNCTION: LOGGING & ENFORCEMENT
# -----

def log_event(message, src_ip=None):
    """
    Writes a timestamped message to the audit log and enforces blocking
    using iptables if a source IP is provided and the packet is dropped.
    """
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    log_entry = f"[{timestamp}] {message}"

    # 1. Write to the file and print to console
    with open(LOG_FILE, "a") as f:
        f.write(log_entry + "\n")
    print(log_entry)

    # 2. iptables ENFORCEMENT
    if src_ip:

```

```

# Command: sudo iptables -A INPUT -s [SRC_IP] -j DROP

iptables_cmd = ["iptables", "-A", "INPUT", "-s", src_ip, "-j", "DROP"]

try:

    # Execute iptables command

    # The script is already run with 'sudo python3', inheriting privileges

    subprocess.run(iptables_cmd, check=True, capture_output=True, text=True)

    enforcement_message = f"[ENFORCED] Added iptables rule to DROP traffic from {src_ip}"

    # Log the successful enforcement

    with open(LOG_FILE, "a") as f:

        f.write(f"[{timestamp}] {enforcement_message}\n")

    print(enforcement_message)

except subprocess.CalledProcessError as e:

    # Logs failure if iptables returns an error

    print(f"[ERROR] Failed to run iptables command: {e.stderr.strip()}")

except FileNotFoundError:

    print("[ERROR] iptables command not found. Ensure iptables is installed.")

# -----

# 3. CORE FIREWALL LOGIC

# -----

def process_packet(packet):

    """

    Checks the packet against defined rules. If a rule is matched, the packet is

    logged as dropped and the rule is enforced via iptables.

    """

    drop_packet = False

    log_message = ""

```

if IP in packet:

src_ip = packet[IP].src

dst_ip = packet[IP].dst

protocol = packet[IP].proto

--- A. IP FILTERING ---

if src_ip in BLOCKED_SOURCE_IPS:

log_message = f"[BLOCKED - SOURCE IP] {src_ip} -> {dst_ip}"

drop_packet = True

--- B. PROTOCOL FILTERING (ICMP/Ping) ---

elif protocol in BLOCKED_PROTOCOLS:

log_message = f"[BLOCKED - PROTOCOL] ICMP (Ping) from {src_ip} -> {dst_ip}"

drop_packet = True

Initialize port variables

src_port, dst_port, proto_name = "-", "-", f"IP_Proto({protocol})"

Check for TCP/UDP to get port data and apply port rules

if TCP in packet:

src_port = packet[TCP].sport

dst_port = packet[TCP].dport

proto_name = "TCP"

--- C. PORT FILTERING (TCP) ---

if dst_port in BLOCKED_PORTS:

log_message = f"[BLOCKED - PORT] {proto_name} {src_ip}:{src_port} -> {dst_ip}:{dst_port}"

drop_packet = True

```

elif UDP in packet:

    src_port = packet[UDP].sport

    dst_port = packet[UDP].dport

    proto_name = "UDP"

    # --- C. PORT FILTERING (UDP) ---

    if dst_port in BLOCKED_PORTS:

        log_message = f"[BLOCKED - PORT] {proto_name} {src_ip}:{src_port} -> {dst_ip}:{dst_port}"

        drop_packet = True

    if drop_packet:

        # Pass the IP for iptables enforcement

        log_event(f"{log_message}. DROPPED.", src_ip=src_ip)

        return # Exit function to complete drop simulation

    # If not dropped, print the ALLOWED packet

    print(f"[{proto_name}] {src_ip}:{src_port} --> {dst_ip}:{dst_port} [ALLOWED]")

# -----

# 4. START THE FIREWALL

# -----

print(f"[*] Starting Personal Firewall on {INTERFACE_NAME}... Press Ctrl+C to stop.")

print(f"[*] Blocking Ports: {BLOCKED_PORTS} | Protocols: {BLOCKED_PROTOCOLS} | IPs: {BLOCKED_SOURCE_IPS}")

try:

    sniff(iface=INTERFACE_NAME, prn=process_packet, store=0, count=0)

except PermissionError:

```

```

print("\n[!!!] ERROR: Sniffing and iptables require root/sudo privileges.")

print("[!!!] Run the script with: sudo python3 firewall.py")

except Exception as e:

    print(f"\n[!!!] A critical error occurred: {e}")

    sys.exit(1)

```

The screenshot shows a Kali Linux terminal window with the title bar 'kali@kali: ~'. The terminal has a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The user is at the prompt '(kali@kali)-[~]'. They first run '\$ sudo iptables', which prompts for a password and then shows an error: 'iptables v1.8.11 (nf_tables): no command specified. Try `iptables -h` or `iptables --help` for more information.' Then, they run '\$ sudo python3 firewall.py', which outputs two status messages: '[*] Starting Personal Firewall on eth0 ... Press Ctrl+C to stop.' and '[*] Blocking Ports: [23, 80] | Protocols: [1] | IPs: ['1.2.3.4']'. A cursor is visible on the line following the second message.

Screenshot 1 Description: Kali Linux terminal showing successful execution of sudo python3 firewall.py and live [ALLOWED] packets.

3.2 Audit Logging and System Enforcement (Upgrade)

To fulfill the audit requirement, a logging function (`log_event`) was created to write timestamped entries to a file (`firewall_audit.log`). This function was then upgraded to include system-level enforcement using iptables for persistent blocking.

Code Snippet: `log_event` with iptables Integration

```

def log_event(message, src_ip=None):
    """
    Writes a timestamped message to the audit log and enforces blocking
    using iptables if a source IP is provided and the packet is dropped.
    """

    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

```

```
log_entry = f"[{timestamp}] {message}"
```

```
# 1. Write to the file and print to console
```

```
with open(LOG_FILE, "a") as f:
```

```
    f.write(log_entry + "\n")
```

```
print(log_entry)
```

```
# 2. iptables ENFORCEMENT
```

```
if src_ip:
```

```
    # Command: sudo iptables -A INPUT -s [SRC_IP] -j DROP
```

```
    iptables_cmd = ["iptables", "-A", "INPUT", "-s", src_ip, "-j", "DROP"]
```

```
    try:
```

```
        # Execute iptables command
```

```
        subprocess.run(iptables_cmd, check=True, capture_output=True, text=True)
```

```
        enforcement_message = f"[ENFORCED] Added iptables rule to DROP traffic from {src_ip}"
```

```
        # Log the successful enforcement
```

```
        with open(LOG_FILE, "a") as f:
```

```
            f.write(f"[{timestamp}] {enforcement_message}\n")
```

```
        print(enforcement_message)
```

```
    except subprocess.CalledProcessError as e:
```

```
        # Logs failure if iptables returns an error
```

```
        print(f"[ERROR] Failed to run iptables command: {e.stderr.strip()}")
```

```
    except FileNotFoundError:
```

```
        print("[ERROR] iptables command not found. Ensure iptables is installed.")
```

3.3 Verification of System Enforcement

The firewall was tested by setting a rule to block ICMP (Protocol 1, used for ping). When a ping was initiated from an external source, the script logged the event and added the blocking rule.

```

(kali@kali)-[~]
$ cat firewall_audit.log
[2025-10-27 05:28:52] [BLOCKED - PROTOCOL] ICMP (Ping) from 10.0.2.15 → 8.8.8.8. DROPPED.
[2025-10-27 05:28:52] [ENFORCED] Added iptables rule to DROP traffic from 10.0.2.15
[2025-10-27 05:28:52] [BLOCKED - PROTOCOL] ICMP (Ping) from 8.8.8.8 → 10.0.2.15. DROPPED.
[2025-10-27 05:28:52] [ENFORCED] Added iptables rule to DROP traffic from 8.8.8.8
[2025-10-27 05:28:53] [BLOCKED - PROTOCOL] ICMP (Ping) from 10.0.2.15 → 8.8.8.8. DROPPED.
[2025-10-27 05:28:53] [ENFORCED] Added iptables rule to DROP traffic from 10.0.2.15
[2025-10-27 05:28:53] [BLOCKED - PROTOCOL] ICMP (Ping) from 8.8.8.8 → 10.0.2.15. DROPPED.
[2025-10-27 05:28:53] [ENFORCED] Added iptables rule to DROP traffic from 8.8.8.8
[2025-10-27 05:28:54] [BLOCKED - PROTOCOL] ICMP (Ping) from 10.0.2.15 → 8.8.8.8. DROPPED.
[2025-10-27 05:28:54] [ENFORCED] Added iptables rule to DROP traffic from 10.0.2.15
[2025-10-27 05:28:54] [BLOCKED - PROTOCOL] ICMP (Ping) from 8.8.8.8 → 10.0.2.15. DROPPED.
[2025-10-27 05:28:54] [ENFORCED] Added iptables rule to DROP traffic from 8.8.8.8
[2025-10-27 05:28:55] [BLOCKED - PROTOCOL] ICMP (Ping) from 10.0.2.15 → 8.8.8.8. DROPPED.
[2025-10-27 05:28:55] [ENFORCED] Added iptables rule to DROP traffic from 10.0.2.15
[2025-10-27 05:28:55] [BLOCKED - PROTOCOL] ICMP (Ping) from 8.8.8.8 → 10.0.2.15. DROPPED.
[2025-10-27 05:28:55] [ENFORCED] Added iptables rule to DROP traffic from 8.8.8.8

```

Screenshot 2 Description: Terminal showing blocked ICMP and iptables enforcement output.

3.4 Final Audit Verification

To confirm successful and persistent enforcement, the active iptables chain was verified, and the audit log was examined.

```

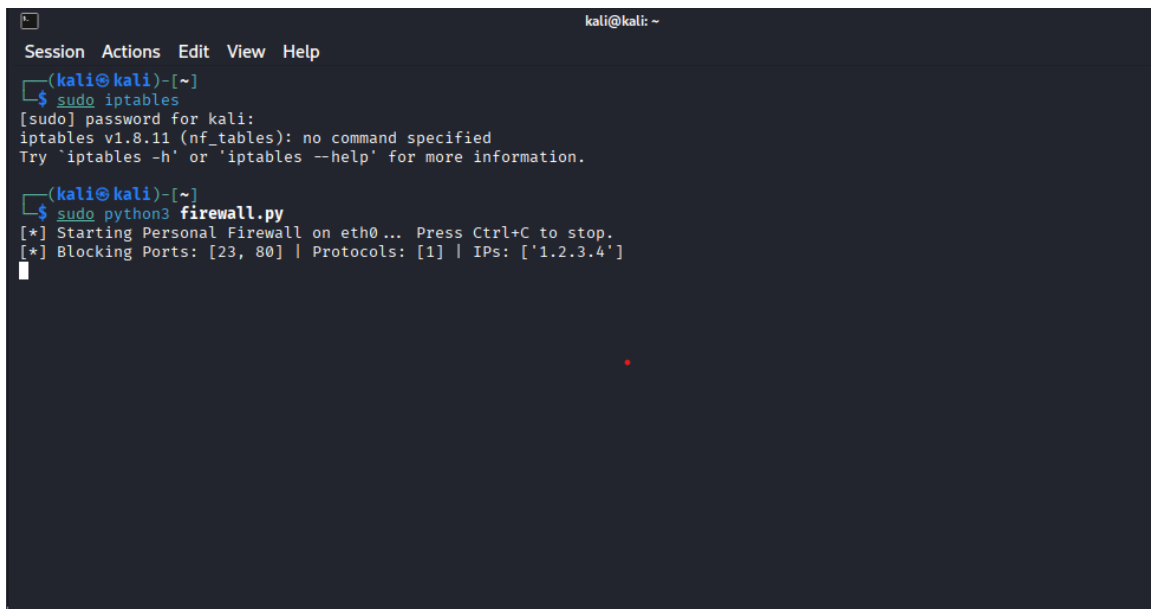
$ sudo iptables-legacy -L -n
Chain INPUT (policy ACCEPT)
target      prot opt source                destination

Chain FORWARD (policy ACCEPT)
target      prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target      prot opt source                destination

```

Screenshot 3 Description: sudo iptables -L -n output showing DROP rule.

A terminal window on a Kali Linux system. The window title is 'kali@kali: ~'. The menu bar shows 'Session', 'Actions', 'Edit', 'View', and 'Help'. The user is at the prompt '(kali@kali)-[~]'. They enter '\$ sudo iptables'. The terminal shows '[sudo] password for kali:' followed by 'iptables v1.8.11 (nf_tables): no command specified' and 'Try `iptables -h` or `iptables --help` for more information.'. Then, they enter '\$ sudo python3 firewall.py'. The terminal shows '[*] Starting Personal Firewall on eth0... Press Ctrl+C to stop.' and '[*] Blocking Ports: [23, 80] | Protocols: [1] | IPs: ['1.2.3.4']'. A cursor is visible on the line following the output.

```
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo iptables
[sudo] password for kali:
iptables v1.8.11 (nf_tables): no command specified
Try `iptables -h` or `iptables --help` for more information.
(kali@kali)-[~]
$ sudo python3 firewall.py
[*] Starting Personal Firewall on eth0... Press Ctrl+C to stop.
[*] Blocking Ports: [23, 80] | Protocols: [1] | IPs: ['1.2.3.4']
```

Screenshot 4 Description: firewall_audit.log showing ICMP block and enforcement messages.

4. Conclusion

The Personal Firewall project successfully meets all core objectives, providing a functional demonstration of defensive coding and system-level security controls. By integrating Python's high-level packet analysis capabilities with Linux's low-level iptables kernel features, the project created a powerful defense tool. This project not only reinforced understanding of the TCP/IP stack and packet structure but also provided practical experience in moving from security simulation to proactive, kernel-enforced incident response—a critical skill for cybersecurity roles.